

NUNCHI · SECURITY REVIEW

Nunchi Review.

A security review of Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Nunchi	Review	Guardian	September 17 – October 7, 2025	October 14, 2025

§ 00

Contents.

39 pages — overview, confidence ranking, scope, 27 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 18 findings	P. 09
	Remediation Review · 9 findings	P. 29
§ 05	Disclaimer	P. 38
§ 06	About Guardian	P. 39

§ 01

Overview.

Guardian reviewed Review across 2 review rounds. The final report includes 4 High, 7 Medium, 6 Low, 10 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	Nunchi-trade/genesis-vaults
CLIENT	METHODOLOGY
Nunchi	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Review	f1c9c4f785c5
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	494f3208c024
AUDIT WINDOW	
September 17 – October 7, 2025	
FINAL REPORT	
October 14, 2025	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	4	0	1	0	3
Medium	7	0	2	0	5
Low	6	0	4	0	2
Informational	10	0	3	0	7
All severities	27	0	10	0	17

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Review in `Nunchi-trade/genesis-vaults`. The scope includes 691 source lines across 9 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/tokens/GenesisVaultSYToken.sol	154	269
src/storage/GlobalStorage.sol	66	129
src/storage/Vault.sol	212	390
src/strategies/BaseGenesisStrategy.sol	19	43
src/strategies/NoYieldStrategy.sol	47	68
src/modules/ConfigurationModule.sol	68	145
src/modules/StrategyModule.sol	34	59
src/modules/VaultModule.sol	61	103
src/libraries/Errors.sol	30	46

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Global Withdrawal Delay For SY Holders	Logical Error	HIGH	RESOLVED	p. 9
H-02	Share Price Inflation Attack	Rounding	HIGH	ACKNOWLEDGED	p. 11
H-03	Cross Vault Drains In No-Strategy Vaults	Logical Error	HIGH	RESOLVED	p. 12
H-04	Cross Vault Drains If Strategy Is Shared	Logical Error	HIGH	RESOLVED	p. 13
M-01	emergencyRecover Function Is Unreachable	Access Control	MEDIUM	RESOLVED	p. 14
M-02	Missing Withdrawal Limit	Validation	MEDIUM	RESOLVED	p. 15
M-03	SY Exchange Rate Mispricing	Logical Error	MEDIUM	RESOLVED	p. 16
L-01	Strategy Yield Distribution Sandwich Attack	MEV	LOW	ACKNOWLEDGED	p. 18
L-02	Missing Event For Critical State Change	Best Practices	LOW	RESOLVED	p. 19
L-03	Harvest Flow May Be Capital Inefficient	Rewards	LOW	ACKNOWLEDGED	p. 20
L-04	Missing Check In validateStrategy	Validation	LOW	ACKNOWLEDGED	p. 21
L-05	Wrong Events In SY Token	Best Practices	LOW	RESOLVED	p. 22
I-01	Unsafe Operations	Best Practices	INFORMATIONAL	RESOLVED	p. 23
I-02	Unused Code	Superfluous Code	INFORMATIONAL	RESOLVED	p. 24

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-03	Variables Not Marked As Immutable	Gas Optimization	INFORMATIONAL	RESOLVED	p. 25
I-04	ConfigurationModule Initialization	Frontrunning	INFORMATIONAL	ACKNOWLEDGED	p. 26
I-05	Overly permissive Fee Bounds	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 27
I-06	Incompatibility With Non-Standard ERC20s	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 28

REMEDIATION REVIEW

M-01	Strategy Switch Resets Performance Fee	Rewards	MEDIUM	ACKNOWLEDGED	p. 29
M-02	DoS Griefing Attack	DoS	MEDIUM	ACKNOWLEDGED	p. 30
M-03	Withdraw Delay Can Be Avoided	Validation	MEDIUM	RESOLVED	p. 31
M-04	Function Is Unreachable	Access Control	MEDIUM	RESOLVED	p. 32
L-01	Mismatch In Migration Waiting Period	Documentation	LOW	ACKNOWLEDGED	p. 33
I-01	Misleading Event In claimStrategyIncentives	Events	INFORMATIONAL	RESOLVED	p. 34
I-02	Unused Code	Superfluous Code	INFORMATIONAL	RESOLVED	p. 35
I-03	Unnecessary Try/Catch Blocks	Superfluous Code	INFORMATIONAL	RESOLVED	p. 36
I-04	Redundant Token Transfer	Best Practices	INFORMATIONAL	RESOLVED	p. 37

HIGH **H-01**

RESOLVED

Global Withdrawal Delay For SY Holders

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/d8b8a6e6245a29cd7b518e709ce8749f3ccea226/src/storage/Vault.sol#L112
REVIEW	Main Review

DESCRIPTION

Users deposit underlying tokens through the SY wrapper, GenesisVaultSYToken. The wrapper calls deposit function on Vault module, where the logic records a cooldown timestamp for the caller address.

```
// Update vault state
vault.sharesOf[user] += shares;
vault.totalShares += shares;
vault.lastDepositTime[user] = block.timestamp;
```

When the SY wrapper deposits, msg.sender is the SY contract address, so the vault stores one lastDepositTime for the entire wrapper. When anyone later redeems via the SY wrapper, the wrapper calls withdraw on VaultModule contract. That path enforces the withdrawal delay against the caller address.

```
// Check withdrawal delay
uint256 nextWithdrawalTime = vault.lastDepositTime[user] +
vault.withdrawalDelay;
if (vault.withdrawalDelay > 0) {
if (nextWithdrawalTime > block.timestamp) {
revert Errors.WithdrawalDelayNotMet(
nextWithdrawalTime,
block.timestamp
);
}
}
```

Because the cooldown is tracked once for the SY address (not per end-user), any deposit via the SY wrapper resets lastDepositTime and pushes out the withdrawal window for every SY holder. An attacker can grief all SY redemptions by repeatedly depositing the minimum allowed amount just before the delay expires, creating a redemption DoS for all users who hold SY for that vault.

H-01 Global Withdrawal Delay For SY Holders

CONTINUED

RECOMMENDATION

Move the cooldown enforcement to the SY wrapper on a per-user basis. Track lastDepositTime[user] inside the SY contract and enforce the delay on redeem. Skip the further withdrawal delay validation for registered SY wrappers.

PROOF OF CONCEPT <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/pull/5/>

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/4>

HIGH **H-02**

ACKNOWLEDGED

Share Price Inflation Attack

CATEGORY	Rounding
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/f1c9c4f785c5471485185b6b81d20039d7287077/src/modules/VaultModule.sol#L23-L49
REVIEW	Main Review

DESCRIPTION

The protocol performs a check that the given user did not receive 0 shares and they have a configurable parameter to set a `minDeposit` amount for vaults. These security checks will decrease impact and likelihood but depending on the configs and the given situation a share price inflation attack may still be possible.

RECOMMENDATION

Be aware and consider to further mitigate against share price inflation attacks by implementing virtual shares or minting shares directly in the init flow of a new vault.

PROOF OF CONCEPT <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/pull/1>

RESOLUTION

Status: Acknowledged.

HIGH **H-03**

RESOLVED

Cross Vault Drains In No-Strategy Vaults

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/d8b8a6e6245a29cd7b518e709ce8749f3ccea226/src/storage/Vault.sol#L301
REVIEW	Main Review

DESCRIPTION

Vaults without a strategy route deposits to the Genesis Factory and price shares using `getTotalAssets`, which, in no-strategy mode, returns the Factory's entire balance of the vault's `liquidityToken`.

```
function getTotalAssets(
    Data storage vault
) internal view returns (uint256) {
    if (address(vault.strategy) == address(0)) {
        return IERC20(vault.liquidityToken).balanceOf(address(this));
    }
    return IGenesisStrategy(vault.strategy).totalAssets();
}
```

Because this balance is shared across all no-strategy vaults that use the same token, a deposit into Vault B increases the Factory's balance and immediately inflates `getTotalAssets` for Vault A. Conversions then use contaminated AUM value, letting redemptions in one vault withdraw assets funded by another, and mispricing new mints.

RECOMMENDATION

Use per-vault custody. Track each vault assets and use it in `getTotalAssets` instead of the Factory balance.

PROOF OF CONCEPT <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/pull/2/>

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/5>

HIGH **H-04**

RESOLVED

Cross Vault Drains If Strategy Is Shared

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Users are able to drain the factory if multiple vaults share the same strategy. The reason for this is that they both use the same summed up total amount for the share price calculation but a different totalShare value.

For example:

- Vault A is connected to Strategy A and 100 WETH is deposited
- Vault B is created and connected to Strategy A
- Both vaults share the same totalAssets amount (balanceOf the strategy contract)
- Eve deposits 1 WETH into Vault B and therefore owns 100% of the shares of Vault B
- Eve withdraws all of her shares and by doing so receives: $\text{assetsOut} = \text{vaultBSharesIn} * \frac{\text{summedUpAssets}}{\text{totalVaultBShares}} = 1e18 * \frac{101e18}{1e18} = 101e18$

— RECOMMENDATION

Consider to enforce a 1 to 1 relationship between vaults and strategies.

PROOF OF CONCEPT <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/pull/4>

— RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/5>

MEDIUM **M-01**

RESOLVED

emergencyRecover Function Is Unreachable

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/f1c9c4f785c5471485185b6b81d20039d7287077/src/tokens/GenesisVaultSYToken.sol#L287-L300>

REVIEW Main Review

DESCRIPTION

The `emergencyRecover` function in the `GenesisVaultSYToken` contract is unreachable as it has a `onlyFactory` modifier. But there is no contract in the factory to call it.

RECOMMENDATION

Consider adding a function to the factor to be able to call it.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/8>

MEDIUM **M-02**

RESOLVED

Missing Withdrawal Limit

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/f1c9c4f785c5471485185b6b81d20039d7287077/src/storage/Vault.sol#L100-L125>

REVIEW Main Review

DESCRIPTION

The `AUDIT_SCOPE.md` document talks about a `Configurable deposit/withdrawal limits` in the `Key Features` of the vault. However only a configurable deposit limit was implemented.

RECOMMENDATION

Consider to implement a configurable withdrawal limit too or adjust the docs accordingly.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/6>

MEDIUM **M-03**

RESOLVED

SY Exchange Rate Mispricing

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/d8b8a6e6245a29cd7b518e709ce8749f3ccea226/src/tokens/GenesisVaultSYToken.sol#L91-L99
REVIEW	Main Review

DESCRIPTION

Users can deposit directly into the Genesis vault (`VaultModule.deposit`) or via the SY wrapper (`GenesisVaultSYToken.deposit`). When depositing via SY, the Genesis factory mints vault shares to the SY contract, and the SY contract mints an equal amount of ERC-20 tokens to the user. The SY token exposes an `exchangeRate` function that indicates the number of underlying assets per SY token.

```
function totalAssets() public view returns (uint256) {
    return IVaultModule(factory).totalAssets(vaultId);
}

/// @notice Get the current exchange rate (assets per SY token)
/// @return rate Exchange rate scaled by 1e18
function exchangeRate() public view override returns (uint256 rate) {
    uint256 _totalSupply = totalSupply();
    if (_totalSupply == 0) {
        return 1e18; // 1:1 for empty vault
    }
    uint256 _totalAssets = totalAssets();
    return (_totalAssets * 1e18) / _totalSupply;
}
```

Because the vault allows direct deposits (EOAs mint vault shares that are not represented by SY supply), `totalAssets` includes all assets, while `SY.totalSupply` includes only SY-backed shares. As soon as non-SY shares are outstanding, the `exchangeRate` becomes inflated. Any off-chain valuation, UI or DEX logic that uses `exchangeRate` as a price proxy will be incorrect, which may open further attack scenarios.

RECOMMENDATION

Compute the SY exchange rate as the price per share, using the vault's `convertToAssets`, so it reflects all outstanding shares (SY + direct holders).

M-03 SY Exchange Rate Mispricing

CONTINUED

```
function exchangeRate() public view override returns (uint256) {  
    uint256 rate = IVaultModule(factory).convertToAssets(vaultId, 1e18);  
    return rate == 0 ? 1e18 : rate;  
}
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/pull/6>

— RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/4>

LOW **L-01**

ACKNOWLEDGED

Strategy Yield Distribution Sandwich Attack

CATEGORY	MEV
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/d8b8a6e6245a29cd7b518e709ce8749f3ccea226/src/modules/VaultModule.sol#L23-L76
REVIEW	Main Review

— DESCRIPTION

A withdraw delay can be configured which reduces the likelihood but it may still be possible and profitable to front a yield distribution in the underlying vault of a strategy and deposit a lot of funds into the vault to gain most of the yield and exit right after the withdraw delay passed. So there may still be enough incentive for MEV here.

— RECOMMENDATION

Be aware and consider one of these options:

- Always configure the withdraw delay accordingly
- Only pick vaults which do not have step-wise jumps in their share price value
- Implement a mechanism that distributes yield over time

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

RESOLVED

Missing Event For Critical State Change

CATEGORY Best Practices
LOCATION Global
REVIEW Main Review

DESCRIPTION

Multiple functions which perform critical state changes do not emit an event:

- `ConfigurationModule::transferOwnership`
- `StrategyModule::setPerformanceFee`

RECOMMENDATION

Consider emitting events in all functions which perform critical state changes to follow best practices.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/commit/2f13bd286f791ade24f545941992578385be34dd>

LOW **L-03**

ACKNOWLEDGED

Harvest Flow May Be Capital Inefficient

CATEGORY Rewards

LOCATION <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/f1c9c4f785c5471485185b6b81d20039d7287077/src/storage/Vault.sol#L494-L515>

REVIEW Main Review

— DESCRIPTION

The harvest flow is the following:

- All of the accrued yield is withdrawn
- The performance fee is collected
- The rest of the yield is deposited back into the vault

This is very capital inefficient if there is a withdraw and or deposit fee in the given vault.

— RECOMMENDATION

Be aware and consider to not use vaults with deposit or withdraw fees or rewrite the logic in that case to only withdraw the performance fee.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Missing Check In `validateStrategy`

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/f1c9c4f785c5471485185b6b81d20039d7287077/src/storage/Vault.sol#L131-L142>

REVIEW Main Review

DESCRIPTION

The `validateStrategy` function does not check if the given strategy contract uses `address(this)` as factory contract. Using a strategy with a different address set as factory could lead to DoS or other major issues.

RECOMMENDATION

Consider implementing this check.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

Wrong Events In SY Token

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/main/src/tokens/GenesisVaultSYToken.sol>

REVIEW Main Review

DESCRIPTION

The `GenesisVaultSYToken` emits events which do not strictly follow the ERC-5115 SY token standard: <https://eips.ethereum.org/EIPS/eip-5115>

RECOMMENDATION

Consider to follow best practices and implement the same events as specified in the ERC-5115 SY Token standard.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/4>

INFORMATIONAL **I-01**

RESOLVED

Unsafe Operations

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/f1c9c4f785c5471485185b6b81d20039d7287077/src/storage/Vault.sol#L485>

REVIEW Main Review

DESCRIPTION

There is a unsafe transfer call in the `collectPerformanceFee` function as well as unsafe approve operations globally.

RECOMMENDATION

Consider always using `safeTransfer` and `safeApprove` functions to follow best practices.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/7>

INFORMATIONAL I-02

RESOLVED

Unused Code

CATEGORY	Superfluous Code
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

There is unused code in multiple parts of the system.

Unused Errors:

- NoFeeRecipient
- NoFeeConfigured
- OnlyRouter
- InvalidAsset
- InvalidRouter
- InsufficientBalance

Unused Imports:

- IGenesisStrategy in the VaultModule contract
- Errors in the GenesisVaultSYToken contract

RECOMMENDATION

Consider to remove unused code.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/7>

INFORMATIONAL I-03

RESOLVED

Variables Not Marked As Immutable

CATEGORY	Gas Optimization
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/d8b8a6e6245a29cd7b518e709ce8749f3ccea226/src/tokens/GenesisVaultSYToken.sol#L22-L28
REVIEW	Main Review

DESCRIPTION

In the GenesisVaultSYToken contract, the vaultId, factory, and asset variables assigned in the constructor are never modified afterward. There are no setter functions or internal assignments that change these values post-deployment. Keeping them as regular public variables causes unnecessary storage reads at runtime, which consume gas, whereas declaring them immutable embeds the values in bytecode at deployment, eliminating storage access costs.

RECOMMENDATION

Declare the mentioned fields as immutable.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/7>

INFORMATIONAL **I-04**

ACKNOWLEDGED

ConfigurationModule Initialization

CATEGORY	Frontrunning
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/d8b8a6e6245a29cd7b518e709ce8749f3ccea226/src/modules/ConfigurationModule.sol#L20
REVIEW	Main Review

— DESCRIPTION

The initialize function, implemented in ConfigurationModule is external and lacks access control. The first caller permanently sets GlobalStorage.owner, after which further calls revert. If deployment and initialization are not atomic, an attacker can front-run the initialization and set themselves as owner, forcing redeployment.

— RECOMMENDATION

Initialize the ConfigurationModule in the Genesis Factory's constructor during deployment to prevent front-running.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-05**

ACKNOWLEDGED

Overly permissive Fee Bounds

CATEGORY	Best Practices
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/d8b8a6e6245a29cd7b518e709ce8749f3ccea226/src/storage/Vault.sol#L157
REVIEW	Main Review

— DESCRIPTION

The `validatePerformanceFee` function only rejects rate greater than 10000 BPS, so a 100% performance fee is currently permitted.

This is an overly permissive configuration that poses unexpected risk if set accidentally.

— RECOMMENDATION

Enforce a sensible hard cap below 100% (e.g. 2000–3000 bps), and consider applying a time lock for fee increases.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

Incompatibility With Non-Standard ERC20s

CATEGORY	Best Practices
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The vault assumes standard ERC-20 behaviour - exact amount is received on transfer, balances change only by that amount and math can safely rely on the requested value. If the underlying token deviates from these assumptions, internal accounting can drift and downstream calls may operate on incorrect balances. Examples include fee-on-transfer or deflationary tokens that deliver fewer tokens than requested, rebasing tokens whose balances change asynchronously or ERC-777 hooks that may introduce the reentrancy risk.

Because the implementation mints shares from the requested amount instead of the observed balance delta, it can over-mint shares and attempt to deposit more into the strategy than it actually holds, breaking internal accounting.

— RECOMMENDATION

Accept only standard ERC-20 tokens or update the vault logic to correctly handle non-standard behaviours. Note that even commonly used tokens such as USDT or USDC could enable transfer fees in the future.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Strategy Switch Resets Performance Fee

CATEGORY Rewards

LOCATION <https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/494f3208c024bb11c7d97e7698fe9560dee0d69c/src/storage/Vault.sol#L369>

REVIEW Remediation Review

— DESCRIPTION

The `switchStrategy` flow calls the `unsetStrategy` function to unset the old strategy. However this function also resets the `performanceFeeRate` and the `feeRecipient`.

This may lead to lost revenue for the protocol.

— RECOMMENDATION

Consider to not reset the performance fee configs or add a boolean if this is wished or not.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

DoS Griefing Attack

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/494f3208c024bb11c7d97e7698fe9560dee0d69c/src/tokens/GenesisVaultSYToken.sol#L210
REVIEW	Remediation Review

DESCRIPTION

There is a withdrawal delay based on when a user was the recipient of a deposit the last time. However an arbitrary address can be set as recipient of a deposit.

This allows users to DoS other users from withdrawing their funds by depositing a small amount of funds and set the victim as recipient.

RECOMMENDATION

As long as the `minDeposit` value is high enough the chance that this happens is unlikely. But be aware and consider to make changes if necessary.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

RESOLVED

Withdraw Delay Can Be Avoided

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/494f3208c024bb11c7d97e7698fe9560dee0d69c/src/tokens/GenesisVaultSYToken.sol
REVIEW	Remediation Review

DESCRIPTION

The withdraw delay can be avoided by transferring the tokens to another EOA and withdrawing with this account instead as the check is performed for the last deposit of the given msg.sender.

RECOMMENDATION

Here are two possible solutions:

- Make withdrawals a 2 step process with a delay in between
- Increase the `lastDepositTime` of the recipient during a transfer and add a `minTransfer` amount to mitigate against another griefing DoS vector

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/16>

MEDIUM **M-04**

RESOLVED

Function Is Unreachable

CATEGORY	Access Control
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/494f3208c024bb11c7d97e7698fe9560dee0d69c/src/strategies/AAVEStrategy.sol#L302-L327
REVIEW	Remediation Review

DESCRIPTION

The `emergencyWithdraw` function in the `AAVEStrategy` contract is unreachable as it has a `onlyFactory` modifier. But there is no contract in the factory to call it.

RECOMMENDATION

Consider adding a function to the factor to be able to call it.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/16>

LOW **L-01**

ACKNOWLEDGED

Mismatch In Migration Waiting Period

CATEGORY	Documentation
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/main/src/modules/MigrationModule.sol#L22
REVIEW	Remediation Review

DESCRIPTION

The migration specification requires a 3-day waiting period before execution, but the implementation sets `MIN_MIGRATION_DELAY = 1 days` and applies `migrationDelay == 0 ? MIN_MIGRATION_DELAY : migrationDelay`. Additionally, GlobalStorage comments state the default is 3 days. This inconsistency can allow migrations to execute earlier than stakeholders expect, reducing the intended review/exit window.

RECOMMENDATION

Align code and documentation by setting MIN_MIGRATION_DELAY to 3 days.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

RESOLVED

Misleading Event In claimStrategyIncentives

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/494f3208c024bb11c7d97e7698fe9560dee0d69c/src/modules/StrategyModule.sol#L136
REVIEW	Remediation Review

DESCRIPTION

The `claimStrategyIncentives` function will pass and emit an `StrategyIncentivesClaimed` event even if no tokens were claimed.

RECOMMENDATION

Consider to revert in that case.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/16>

INFORMATIONAL **I-02**

RESOLVED

Unused Code

CATEGORY	Superfluous Code
LOCATION	Global
REVIEW	Remediation Review

DESCRIPTION

There is unused code in multiple parts of the system:

- `IERC165` import in the `ConfigurationModule`
- `validateVaultInteraction` function in the `GlobalStorage` library
- `InsufficientBalance` error in the `AAVEStrategy` contract
- `safeTransfer` in the catch flow of the `AAVEStrategy` `deposit` function is redundant as the state will reset anyway on a revert

RECOMMENDATION

Consider to remove redundant code.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/16>

INFORMATIONAL **I-03**

RESOLVED

Unnecessary Try/Catch Blocks

CATEGORY	Superfluous Code
LOCATION	AAVEStrategy.sol
REVIEW	Remediation Review

DESCRIPTION

AAVEStrategy contract uses try/catch around Aave calls in deposit, withdraw, and withdrawAll functions, where failures should revert the whole transaction. This masks root causes by replacing Aave's revert data with generic errors and adds bytecode size and complexity with no functional benefit.

RECOMMENDATION

Consider removing try/catch pattern from deposit, withdraw and withdrawAll functions.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/16>

INFORMATIONAL **I-04**

RESOLVED

Redundant Token Transfer

CATEGORY	Best Practices
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1758024587975/blob/494f3208c024bb11c7d97e7698fe9560dee0d69c/src/strategies/AAVEStrategy.sol#L109
REVIEW	Remediation Review

DESCRIPTION

In AAVEStrategy.deposit, after pulling tokens from the factory, the catch block calls safeTransfer and then reverts. Since the revert rolls back all state changes in this transaction, the transfer is redundant and adds an unnecessary external call.

RECOMMENDATION

Remove the safeTransfer in the catch block and revert directly.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/16>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.